



Console Based Notepad Application Searching and Suggestion

Instructions:

1. **Read the assignment carefully** before starting to implement your solution.
 2. **Deadline:** Submit your work by November 11th, 11:55 PM.
 3. **Part 1: Searching**
Implement a searching feature that allows users to quickly find specific data within the file. The time complexity should not exceed $1.5 * \log(n)$, utilizing an N-Ary tree for data storage.
 4. **Part 2: Suggestions**
Implement a word suggestion feature that provides users with options for completing their sentences based on previously entered data. This will involve creating a "Chilli Milli Tree" for efficient word completion suggestions.
 5. **Testing:** Ensure your program functions correctly and meets the complexity requirements.
 6. **Formatting:** Follow proper coding standards and include comments where necessary to explain your code.
-

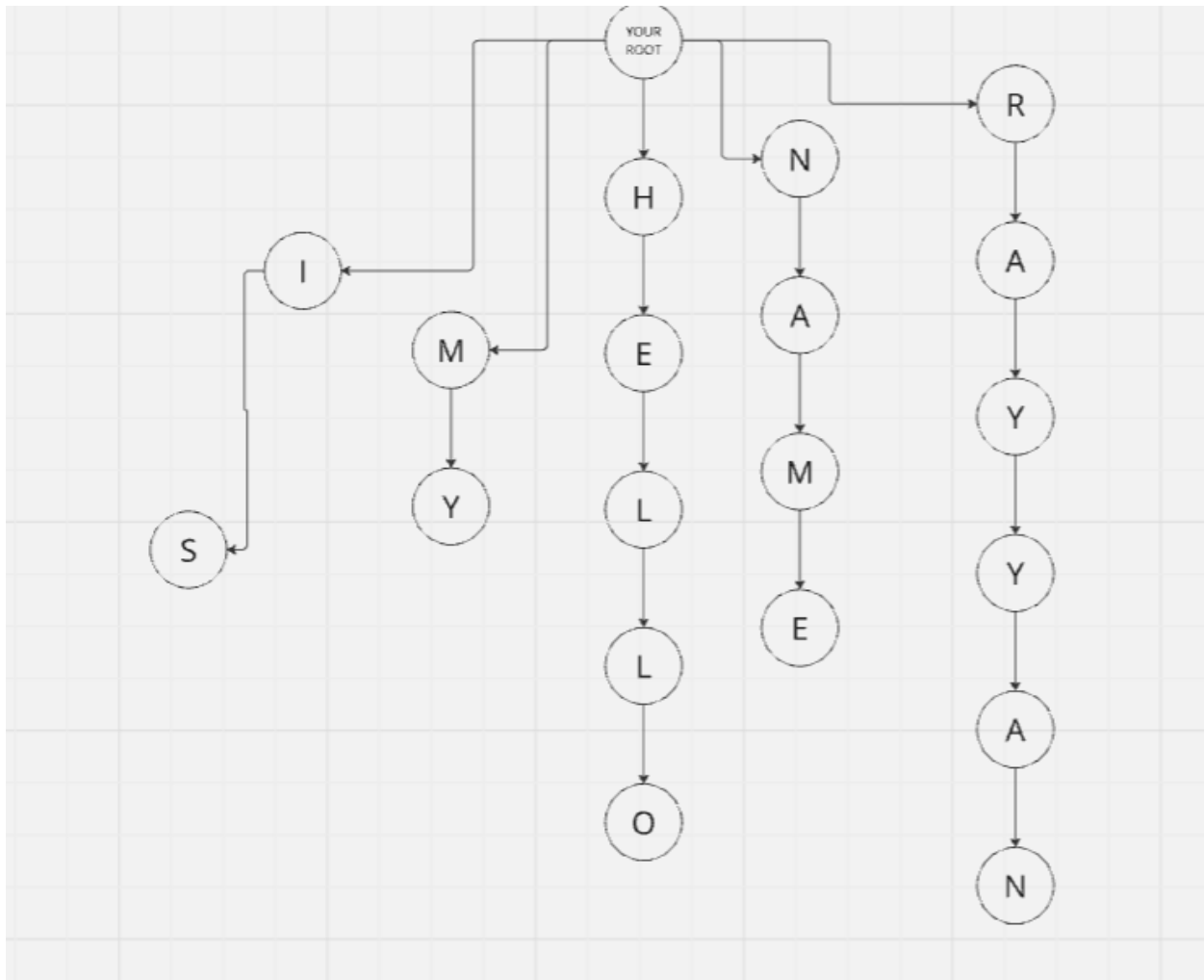
Enhance your Console-Based Notepad Application by implementing the following additional features:

Part 1: Searching

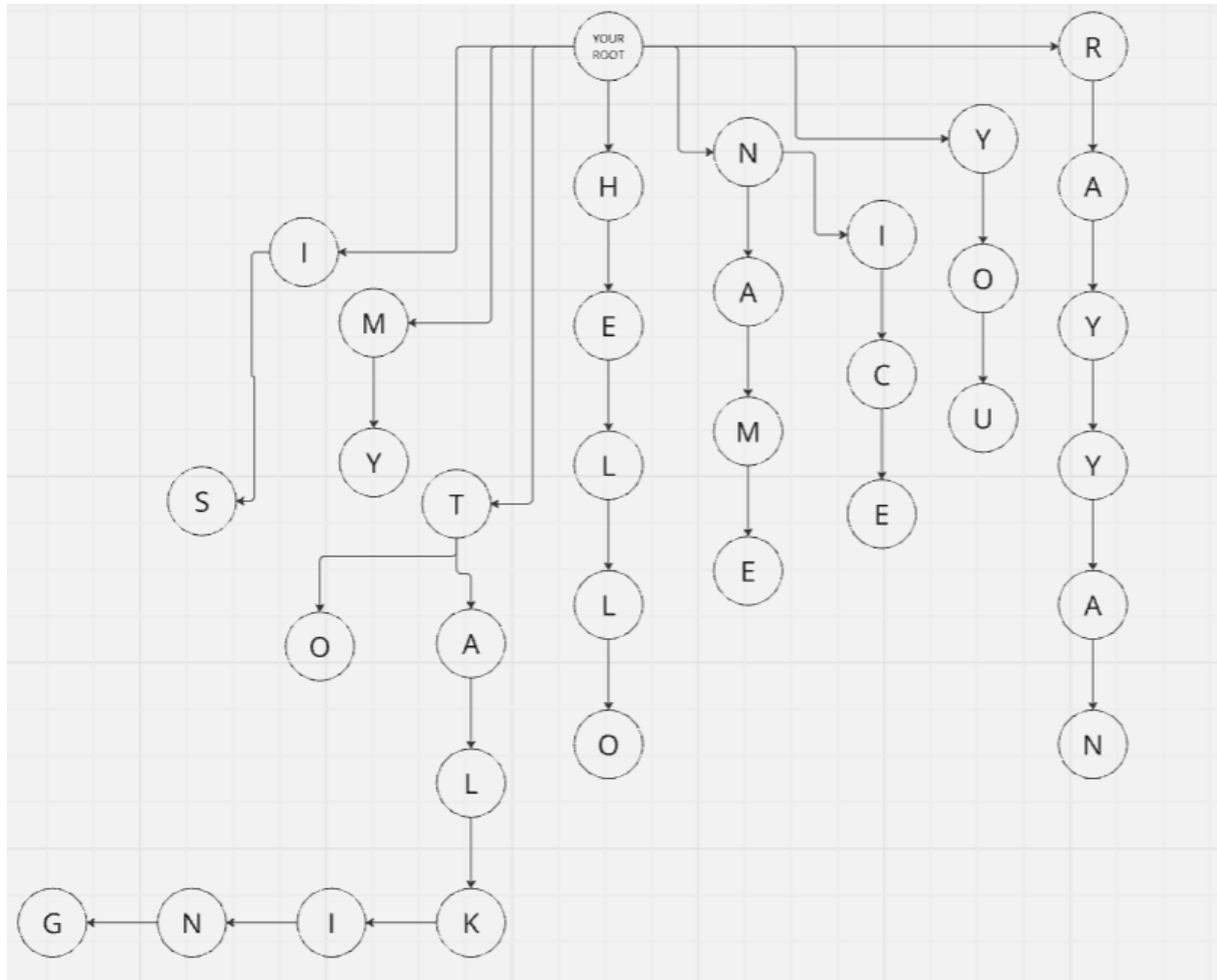
The user wishes to search for a specific data in the file, however it is taking a large amount of time. You need to reduce the time it takes to search for a given data i.e the time complexity should not exceed $1.5 * \log(n)$. You are required to use a N-Ary tree (in addition to the work you did in Assignment 2) to save the user data in order to help them in searching.

How will the data be saved?

For instance the user typed *"Hello my name is rayyan."*

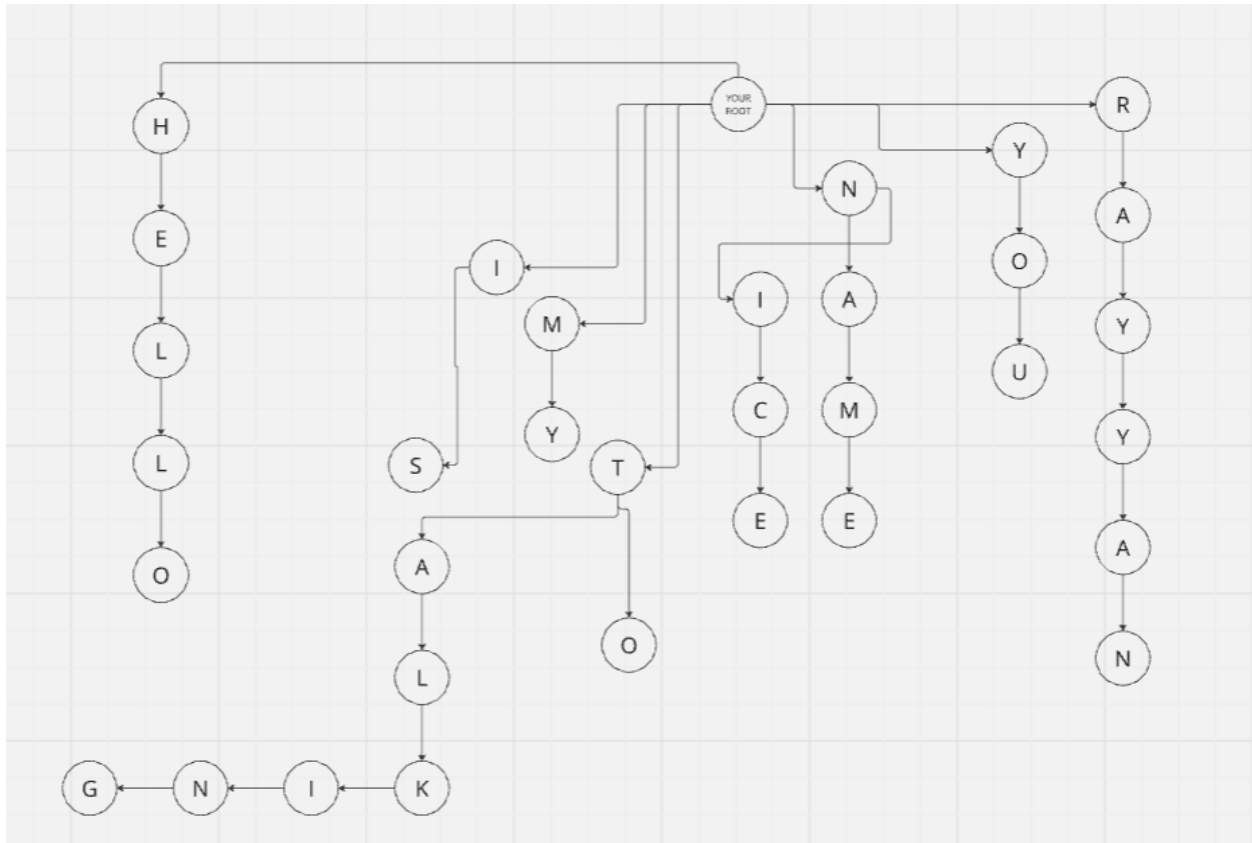


Now for instance the user enters ahead. *"Nice talking to you Rayyan."*
This is how the data will be saved now in the N-ary tree.



Each node must be dynamically increasing and can have a maximum of 26 children, corresponding to the number of alphabets.

It may be noted that the N-ary tree may be in any format with different students having different N-ary trees for instance, another example of the N-ary tree would be.



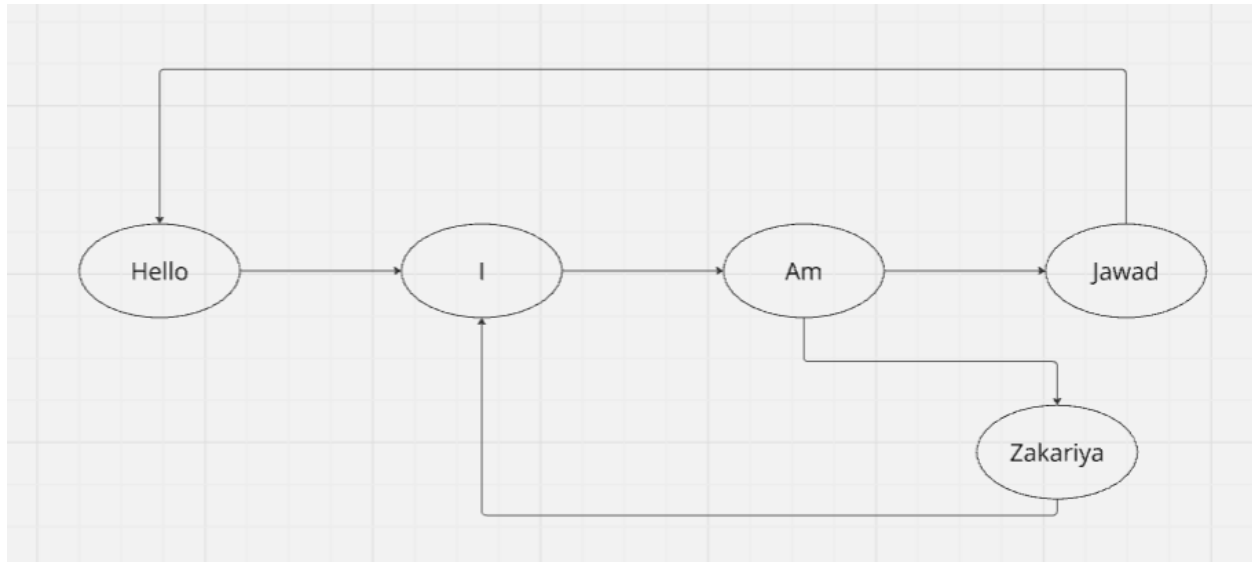
In this diagram drawn, the tree created is slightly different although the final output would remain the same.

Part 2: Suggestions

While the user is typing, he might need suggestions for finishing a sentence. For that purpose, you must implement a word suggestion based on the user's previous entered data. Although it primarily uses NLP, you are required to do a similar basic task using a special type of tree which we would be calling a **chilli milli tree**.

How will the chilli milli tree save the data?

Let's assume user entered the text *"Hello I am Jawad Hello I am Zakariya I am"*



Requirements (what would be expected from you):

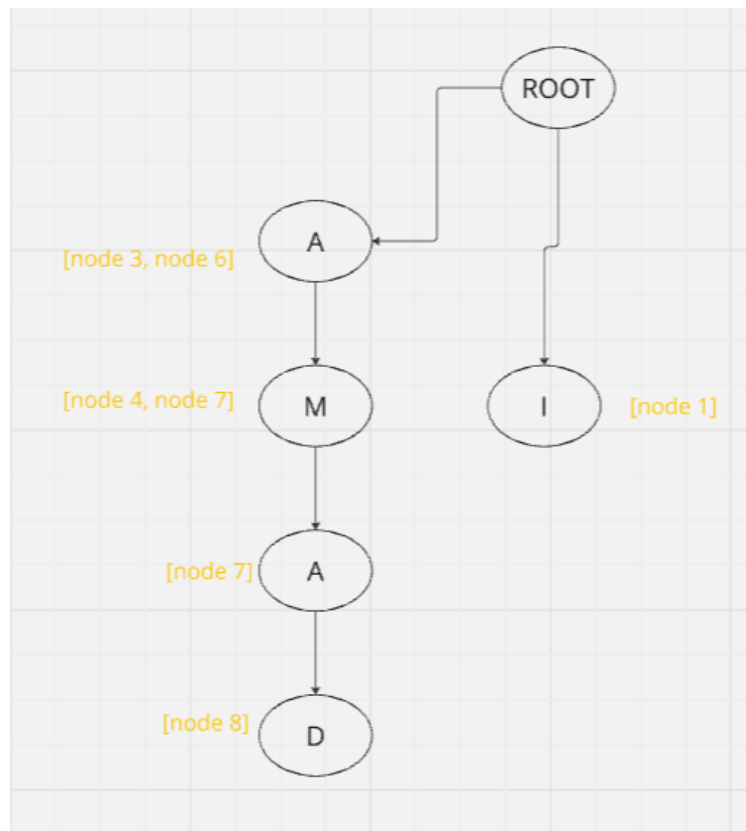
- 1) **Creating the N-ary tree and chilli milli tree from a data:** User enters the data and an N-ary tree and chilli milli tree is created.
- 2) **Searching for a text(for both single word or a sentence):** You would traverse the N-ary tree and print out the line numbers(s) where the data is found. You are not supposed to traverse the data/linkedlist for searching which you created in Assignment 2. You should know the line numbers of a character and must display the line number of the first character of the (word/sentence). Moreover, if the substring exists, highlight by changing the color of text for all instances of the substring found in text.

For doing that one way is to do chaining, to save the nodes which occur for that character, you may use any method you like, for example.



The above is the data which you implemented in Assignment 2

The N-ary tree would be saving the nodes where that specific character is arriving in the text.



For the above example if a user asks for searching and enters “AM” for searching, all possible places where “AM” is occurring must now be colored and displayed automatically.

Your program should work for searches of more than 1 word separated by spaces. For example, a user has written “Hello my name is hania. This is an easy assignment”. Now if the user searches “is” the text becomes “Hello my name *is* hania. This *is* an easy assignment.”. If the user searches “is hania” the text becomes “Hello my name *is hania*. This is an easy assignment”

Display search results in the "Search Area" on the right-hand side of the screen. The search results should include line numbers and highlight the matched word(s) within the text area. For colorizing, you



may use anything you like. (Hint: Add a new data member in the class Node which stores the character, through which you can keep track if you need to color a specific character or not).

Ensure that the search is case-insensitive and able to handle multiple occurrences of the same word

NOTE: The complexity MUST not exceed $1.5 \cdot \log(n)$.

3) **Word completion:** For completing the word, you are supposed to use the N_ary tree.

For example, a user has written *"Hello, my name is Rayyan. Nice talking to you Rayyan."*. Next the user writes *"Good t@"*. (@ signifies the user is requesting for word completion). Users should get suggestions: *"talking"* and *"to"*. Display the suggestions in the "Suggestions Area" and allow the user to select one of the words using keyboard shortcuts to insert it at the current cursor position

NOTE: The complexity MUST not exceed $O(n)$.

4) **Sentence completion:** For sentence completion you are supposed to use a chilli milli tree.

After the user types a sentence or phrase, the system should search through the text and suggest the most frequent words that follow the last word of the sentence. Display these suggestions in the "Suggestions Area" for the user to choose from and insert.

For example a user previously entered *"Hello I am Jawad Hello I am Zakariya I am "*. The user would get the suggestions: *"Jawad"* as well as *"Zakariya"*. If the user then further writes *"Jawad *"*, (* signifies the user is requesting for sentence completion) in that case the user should get the suggestion *"Hello "*. This is based on the data the user has already entered previously. If for instance your program cannot give any suggestions, you may display *"no words found"*.

NOTE: The complexity MUST not exceed $O(n)$.

- **Case sensitivity doesn't matter for the scope of this assignment.**